Ch 04: Traps and System Calls

xv6 a simple Unix-like teaching Operating System

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

EAST TEXAS A&M

Introduction Traps and System Calls (1 of 3)

There are 3 kinds of event which cause the CPU to set aside ordinary execution of instructions and force a transfer of control to special kernel code that handles the event:

1. **system call**: a user program executes the `ecall` (RISC-V) instruction to ask the kernel to do something for it
2. **exception**: an instruction (user or kernel mode) does something illegal, such as attempt to load an invalid virtual address
3. device **interrupt**: when a device signals that it needs attention

EAST TEXAS A&M

Introduction Traps and System Calls (2 of 3)

- We will use **trap** as a generic term for all 3 situations
- In all cases, whatever code was executing at the time of a trap will later need to resume
  - Trap call needs to be **transparent**, interrupted code does not know it happened
- A trap forces transfer of control into the kernel
  - kernel saves registers and other state (PC) so can resume execution later
  - kernel executes appropriate handler code (system call, exception handler or device driver)
  - kernel restores the saved state and returns from trap once handled, original code (might) resume where it left off

Introduction Traps and System Calls (3 of 3)

xv6 trap handling proceeds in four? stages:

1. hardware actions taken by the RISC-V cpu
2. some assembly instructions that prepare the way for kernel c Code trap handlers
3. a C function that decides what to do with the trap
4. finally the system call or device-driver service routine
   - implicitly state is saved in 1,2, and restored at end of 4

Kernel code that processes a trap is called a **handler**

The first handler instructions are usually written in assembler and are sometimes called a **vector**

EAST TEXAS A&M

RISC-V Trap Machinery

RISC-V CPU has a set of hardware control registers that the kernel writes to tell the CPU how to handle traps, and that the kernel can read to find out about a trap that has occurred

- **stvec**: kernel writes address of its trap handler code here, RISC-V CPU jumps to the address in `stvec` to handle a trap
- **sepc**: when a trap occurs, RISC-V saves the program counter here
  - `stret` (return from trap) copies `sepc` to `pc`, kernel can write `sepc` to control where `sret` goes
- **scause**: RISC-V puts a number here that describes the reason for the trap
- **sscratch**: kernel trap handler code uses `sscratch` to help avoid it overwriting user registers before saving them
- **sstatus**: The `SIE` bit in `sstatus` controls whether device interrupts are enabled. If kernel clears `SIE` RISC-V will defer device interrupts until it is set again.
  - The `SPP` bit indicates whether a trap came from user mode or supervisor mode, and controls what mode `sret` returns to.

EAST TEXAS A&M

RISC-V Hardware Actions

When it forces a trap, the RISC-V hardware does the following

1. If the trap is a device interrupt, and the `sstatus` `SIE` bit is clear, don't do any following
2. Otherwise, disable interrupts by clearing the `SIE` bit in `sstatus`
3. Copy the `pc` to `sepc`
4. Save the current mode (user or supervisor) in the `SPP` bit in `sstatus`
5. Set `scause` to a number indicating the trap's cause
6. Set the mode to supervisor
7. Copy `stvec` to the `pc`
8. Start executing at the new `pc`

These actions are hardwired into the CPU, but note RISC-V doesn't do the following, which are sometimes done on other hardware architectures:

- Does not switch to the kernel page table (so will still be using user/process page table)
- Doesn't switch to a stack in the kernel (so still using user stack initially)
- Doesn't save any registers, other than the `pc`

Traps from User Space

xv6 handles traps differently depending on whether the trap occurs while executing in kernel or in user code

A trap may occur for any of the 3 reasons from user space, program makes a system call (`ecall` instruction), does something illegal, or a device interrupts.

- `stvec` is set to jump to `uservec` in the trampoline page (all address spaces map here)
  - saves all registers in TRAPFRAME
  - switches to the kernel page table
- calls into `usertrap()`
  - determines and handles system call, device interrupt or exception
- `prepare_return()` and ultimately `userret` are called in trampoline, restores registers and returns to user mode

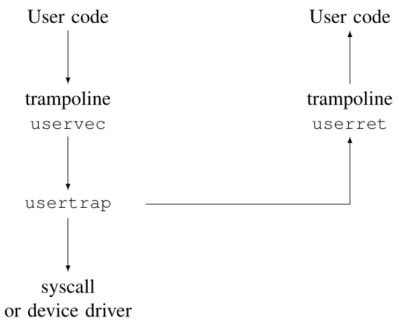

Figure 1: Outline of how a trap from user code is handled (Cox et al., 2026, pg.45)

EAST TEXAS A&M

Calling System Calls (1 of 2)

- User programs call library functions in order to make system calls, for example

```
write(2, "$ ", 2); // write stdout fd=2, the string, length 2 chars
```

- You can find the library function in `user/usys.S`:

```
write:
  li a7, SYS_write
  ecall
  ret
```

- The 3 parameters to write are passed in registers a0, a1 and a2
  - a7 holds the system call number, it is set by the call to write as shown
- The `ecall` instruction traps from user space causing `uservec` `usertrap()` then `syscall()` to execute

Calling System Calls (2 of 2)

- `syscall()` retrieves the system call number from `a7` and uses to index into `syscalls`
  - for example `SYS_write` indexs and calls the `sys_write()` function in teh kernel
- When `sys_write()` returns, `syscall` records its return value in `p->trapframe->a0`
  - This will be what the original user-space call to `write()` will be returned
  - By convention 0 is success, and non zero is failure? (I disagree with book a bit here).

EAST TEXAS A&M

System Call Arguments

- System call arguments start out in user registers `a0` `a1` ... and are moved to the trap frame by the kernel trap code
- kernel functions `argint()` `argaddr()` and `argfd()` canbe used to retrieve the n'th system call argument from the trap frame as an integer, pointer, or a file descriptor
- Sometimes a pointer is passed, will be a pointer to user-space address space
  - Has to be checked, could be buggy or malicious
  - Also the kernel page table mappings are not the same as the user page table, so kernel cannot use ordinary instructions to load or store from user-supplied addresses
- kernel implements functions that safely transfer data to and from user-supplied addresses
  - `fetchstr()` retrieve string file-name arguments
  - `copyinsstr()` copy in str copies up to `max` bytes to `dst` from virtual address `srvca` in the user page table `pagetable`

Traps from Kernel Space

- Traps while in kernel mode are handled in a different way than traps from user space
- `stvec` will point to `kernelvec`
  - Can rely on only executing when already in kernel mode, so don't need to switch kernel page table nor kernel stack
  - `kernelvec` saves registers needed on the kernel stack (rather than trapframe) which makes sense because
    - also this allows for a trap to happen when already in a trap, since pushing on stack
    - also for multicore, this allows to handle traps that occur on different cpus
- Only device interrupts are handled as traps from kernel traps.
  - Obviously we don't need system calls, that is a mechanism to switch from user space to kernel space
  - Exceptions here will just cause a kernel `panic()`, they cannot be recovered from as an exception from user program might sometimes be able to

Summary

References

Cox, R., Kaashoek, F., & Morris, R. (2026). *Xv6: A simple, unix-like teaching operating system*. Retrieved September 2, 2025, from https://github.com/mit-pdos/xv6-riscv-book

EAST TEXAS A&M